



Livrable

Data Pipelines for IA

Scraper books.toscrape.com

Auteur	Victorien Alleg
Enseignant	Matthieu Larboullet
Promotion	Eugenia School — 2025/2026
Session	Session 3 — Optimisation & Docker
Date	Mai 2026

Sommaire

1. Introduction

- 1.1. Contexte
- 1.2. Problématique

2. Analyse du site et des données

- 2.1. Description du site
- 2.2. Structure des données ciblées

3. Code principal

4. Problématiques rencontrées

5. Analyse des données & Visualisations

6. Conclusion

- 6.1. Réponse à la problématique
- 6.2. Limites et améliorations possibles

1. Introduction

1.1. Contexte

Dans le cadre du cours **Data Pipelines for IA** à Eugenia School, l'objectif de la Session 3 est de concevoir un pipeline de scraping web optimisé, conteneurisé sous Docker, et supervisé par des logs professionnels. Le projet s'inscrit dans une progression pédagogique en trois sessions : Session 1 (scraping basique), Session 2 (Docker), Session 3 (optimisation & monitoring).

Ce livrable documente l'intégralité du travail réalisé : architecture technique, choix d'optimisation, résultats mesurés et mise en production via Docker et GitHub.

1.2. Problématique

Comment scraper l'intégralité d'un site web (1 000 fiches, 50 pages catalogue) de manière **fiable, rapide et automatisée**, tout en garantissant :

- La résistance aux pannes réseau (retry, backoff exponentiel)
- La visibilité en temps réel sur l'état du scraping (logs structurés)
- La reproductibilité sur n'importe quelle machine (Docker)
- Un temps d'exécution inférieur à 11 secondes pour 1 050 requêtes

2. Analyse du site et des données

2.1. Description du site

books.toscrape.com est un site de démonstration conçu pour l'apprentissage du scraping web. Il contient un catalogue de **1 000 livres** répartis sur **50 pages catalogue** (20 livres par page), chaque livre ayant sa propre fiche détaillée.

Caractéristique	Valeur
URL	https://books.toscrape.com
Pages catalogue	50 pages (page-1.html → page-50.html)
Livres au total	1 000 fiches
Requêtes HTTP totales	1 050 (50 catalogue + 1 000 fiches)
Technologie	HTML statique — pas de JavaScript
Pagination	URLs prévisibles et séquentielles

2.2. Structure des données ciblées

Chaque fiche livre contient les informations suivantes :

Champ	Source HTML
title	Balise <h1>

price	<p class='price_color'>
rating	Classe CSS de <p class='star-rating'> (One → Five)
category	3ème élément du fil d'Ariane <ul class='breadcrumb'>
date	Date du jour (générée au moment du scraping)

3. Code principal

Architecture en 2 phases parallèles

L'optimisation centrale du projet est la **parallélisation complète** des deux phases du pipeline. Contrairement à une approche séquentielle classique (1 requête à la fois), le scraper envoie des dizaines de requêtes simultanément grâce à un **ThreadPoolExecutor**.

	Méthode	Requêtes	Durée mesurée
	50 URLs en parallèle	50	~1.6s
	1000 URLs en parallèle	1 000	~8.6s
	50 workers simultanés	1 050	< 11s

Optimisation clé : parallélisation de la Phase 1

Dans la version naïve (Session 1), le catalogue était parcouru séquentiellement : on télécharge page-1, lit le bouton 'Suivant', télécharge page-2, etc. Durée : **~8.6 secondes** pour 50 pages.

L'insight clé : les URLs sont **prévisibles** (page-1.html ... page-50.html). On peut donc les générer à l'avance et les télécharger toutes simultanément. Durée après optimisation : **~1.6 secondes**. Gain : ×5.

Connection pooling avec requests.Session

Sans session HTTP, chaque requête ouvre une nouvelle connexion TCP (+100ms/requête). Sur 1 000 requêtes, cela représente 100 secondes perdues. L'**HTTPAdapter** configure un pool de connexions réutilisées (HTTP keep-alive), dimensionné au nombre de workers.

Compteur de requêtes thread-safe

Un compteur global `_request_count` protégé par un **threading.Lock** comptabilise chaque requête HTTP en temps réel. Sans verrou, deux threads pourraient lire la même valeur et produire des résultats incorrects (race condition).

Gestion des erreurs — Backoff exponentiel

Type d'erreur	Comportement
ConnectionError / Timeout	Retry : 2s → 4s → abandon
HTTP 429 (rate limit)	Attente longue (×2) + retry
HTTP 404 / SSL / Redirections	Skip immédiat (pas de retry)

SIGTERM (docker stop)	Arrêt propre, données sauvegardées
Ctrl+C (SIGINT)	Arrêt propre, données sauvegardées

4. Problématiques rencontrées

1. Docker logs vides

Avec CMD ["cron", "-f"], les logs générés par les scripts cron ne remontaient pas dans docker logs. Solution : entrypoint.sh lance cron en arrière-plan puis exécute tail -f sur le fichier de log, redirigeant ainsi le fichier vers stdout du container.

2. Accumulation des données CSV

Entre deux runs, le fichier books.csv s'accumulait au lieu d'être réinitialisé. Cause : init_csv était appelé en Phase 2 (après ~1s) et non au démarrage. Solution : appel de init_csv en mode 'w' (écrase) à la toute première ligne du pipeline.

3. Phase 1 séquentielle — goulot d'étranglement

La navigation page par page du catalogue prenait 8.6s car chaque requête attendait la réponse pour connaître l'URL suivante. Solution : génération directe des 50 URLs (page-1.html à page-50.html) et téléchargement parallèle simultané.

4. Race condition sur le compteur de requêtes

Avec plusieurs threads écrivant simultanément sur _request_count, des valeurs incorrectes pouvaient apparaître. Solution : threading.Lock garantit l'atomicité de chaque incrément.

5. Connectivité Docker — socket introuvable

Sur macOS avec Docker Desktop, la socket Docker n'est pas au chemin standard. Solution : préfixer les commandes avec DOCKER_HOST=unix:///.../.docker/run/docker.sock.

5. Analyse des données & Visualisations

À chaque exécution, le pipeline scrape l'intégralité des 1 000 livres du site et les exporte dans **books.csv**. Voici les statistiques de performance mesurées lors des tests de validation :

Métrique	Résultat mesuré
Requêtes HTTP totales	1 050 (50 catalogue + 1 000 fiches)
Livres collectés	1 000 / 1 000 (100% de succès)
Erreurs	0
Durée — Phase 1 (catalogue)	~1.6s (parallèle, 50 workers)
Durée — Phase 2 (fiches)	~8.6s (parallèle, 50 workers)
Durée totale	< 11 secondes ✓
Débit	~102 req/s
Taille du CSV	~85 Ko (1 001 lignes)

Comparaison avant / après optimisation

	Méthode	Durée site complet
1 (naïve)	Séquentiel, 1 req à la fois	~3 minutes
3 v1 (8 workers)	Phase 1 séquentiel + Phase 2 parallèle	~42 secondes
3 v2 (20 workers)	Phase 1 seq. + Phase 2 parallèle	~19 secondes
3 final (50 workers)	Phases 1 ET 2 entièrement parallèles	< 11 secondes ✓

Structure du fichier books.csv

Colonnes : title, price, rating (1-5), category, date

```
title,price,rating,category,date
A Light in the Attic,£51.77,3,Poetry,2026-05-19
Tipping the Velvet,£53.74,1,Historical Fiction,2026-05-19
Soumission,£50.10,1,Fiction,2026-05-19
Sharp Objects,£47.82,4,Mystery,2026-05-19
...
```

6. Conclusion

6.1. Réponse à la problématique

L'objectif initial était de scraper 1 000 livres de manière fiable, rapide et automatisée. Le pipeline développé répond à ces trois critères :

- **Fiable** : 100% de succès sur tous les tests, gestion complète des erreurs réseau, arrêt propre sur SIGTERM.
- **Rapide** : 1 050 requêtes traitées en < 11 secondes grâce à la parallélisation complète des deux phases.
- **Automatisé** : container Docker avec cron toutes les minutes, logs visibles via docker logs, GitHub Actions pour CI quotidien.

Le gain de performance entre la version naïve (Session 1) et la version finale est d'un facteur **×18** (3 minutes → 10 secondes), obtenu grâce à la parallélisation, le connection pooling HTTP, et l'élimination du goulot d'étranglement de la Phase 1.

6.2. Limites et améliorations possibles

- Paralléliser davantage : augmenter les workers à 100+ (limité par le serveur cible)
- Ajouter un export en base de données (PostgreSQL, MongoDB) en plus du CSV
- Implémenter un dashboard de monitoring en temps réel (Grafana + logs structurés JSON)
- Ajouter des notifications Slack/Discord en cas d'échec (webhook)
- Versionner les données : conserver l'historique des scrapes pour détecter les changements de prix
- Tester avec Scrapy pour comparer les performances sur des sites plus complexes (JS rendu)